SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO2: *Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 25

Duration :60 Mins
On Class
Total Marks:50

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I H.VAISHNAV(192524251) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: H.VAISHNAV(192524251)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

M. VAISHNAV

CO2 - AT-2 (Compiler Design) 192522251

Q1. Parser validation of id + id * id

The parser checks whether the token sequence follows the grammar rules. It recognizes id * id first because * has higher precedence, then combines it with id +.

Hence, the expression is syntactically valid.

Q2. Derivation for aaabbb

Grammar: $S \rightarrow aSb \mid \varepsilon$

Derivation :-

S
$\Rightarrow aSb$
$\Rightarrow aaSbb$
$\Rightarrow aaaSbbb$
$\Rightarrow aaabbb$

∴, aaabbb belongs to the language.

Q3. Remove left Recursion.

Given,

$E \rightarrow E + T \mid T$

After removing left Recursion :-

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

Q4. First (A)

Grammar :-

$A \Rightarrow aB \mid \varepsilon$

$B \Rightarrow B$

First (A) = { a, ε }

Q5. Suitability for top - Down parsing.

Grammar :-

$A \rightarrow Aa \mid b$

It contain left Recursion,

So it is not suitable

Q6. left Factoring :-

Given,

$S \rightarrow$ while E do S

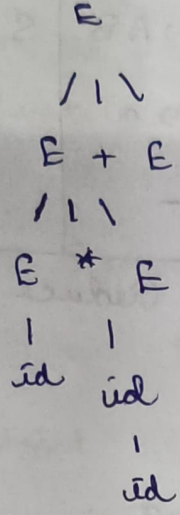
| while E do S else S

left factored grammar :-

$S \rightarrow$ while E do SS'

$S' \rightarrow$ else S | ε

Q7. Parse tree for id * id + id



Q8. Recursive Descent Procedures

```
void S()
{  match ('a');
   A();
}
void A()
{
   if (input == 'b')
   {
      match ('b');
      A();
   }
   else if (input == 'c')
      match ('c');
}
```

Q9. Predictive Parsing Table :-

Grammar :-

$S \rightarrow AB$, $A \rightarrow a \mid \varepsilon$, $B \rightarrow b$

Q17. closure of LR(0) item

Starting item :-

$S' \rightarrow .S$

closure :-

$S' \rightarrow .S$

$S \rightarrow .aA$

Q18. Ambiguity check :-

$E \rightarrow E + E \mid id$

String :-

id + id + id

can be parsed as :-

(id + id) + id

or

id + (id + id)

Q19. Leftmost Derivation

$E \Rightarrow E + E$

$E \Rightarrow id + E$

$E \Rightarrow id + E * E$

$E \Rightarrow id + id * E$

$E \Rightarrow id + id * id$

Q20. Left Factoring

$S \rightarrow aBS' \mid b$

$S' \rightarrow C \mid d.$

Q21. First(F)

First(F) :-

{ C, id }

Q22. Predictive Parsing Stability

$S \rightarrow aA \mid aB$

$A \rightarrow b$

$B \rightarrow c$

Not suitable

First(aA) = {a}

First(aB) = {a}

Q23. Recursive

```
void S()
  { if (input == 'a')
    {
      match('a');
      match('b');
       S()
    }
  }
```

Q25. Predictive Parsing Table :-

Non-Terminal	a	b
S	S→aS	S→b

∴ It is the end